

PRAKTIKUM MACHINE LEARNING

Bab 5

Decision Tree: Algoritma CART/ID3 dan Teknik Pruning

Kumpulan Praktikum Machine Learning

Nama	Ahmad Dandi Subhani
NPM	202343500126
Kelas	R7B
Mata Kuliah	Machine Learning
Dosen	Nurfidah Dwitianti, M.Si.

2 Oktober 2026

Daftar Isi

Decision Tree: Algoritma CART/ID3 dan Teknik Pruning	2
Ringkasan Konsep	3
5.12 Latihan Praktikum (Modul Bab 5)	4
Percobaan Mandiri	12
Kesimpulan Bab 5	18

Tools yang Digunakan

Python 3.12.3 • **numpy** 2.5.3 • **pandas** 3.0.6 • **matplotlib** 3.11.2 • **seaborn** 0.13.2 • **scikit-learn** 1.9.1 • **imbalanced-learn** 0.14.2

Praktikum Machine Learning: Bab 5

Decision Tree: Algoritma CART/ID3 dan Teknik Pruning

Nama	Ahmad Dandi Subhani
NPM	202343500126
Kelas	R7B
Mata Kuliah	Machine Learning
Dosen	Nurfidah Dwitianti, M.Si.

Catatan: notebook ini saya jalanin di **Google Colab / Jupyter Notebook**. Dataset yang saya pake: Breast Cancer Wisconsin dari `sklearn.datasets` (569 pasien, 30 fitur), filenya `data/breast_cancer.csv`. Jadi nggak perlu unduh file eksternal.

Ringkasan Konsep

Decision Tree (pohon keputusan) itu model klasifikasi yang kerjanya ngebagi data secara **rekursif (recursive binary splitting)**: mulai dari root node, tiap internal node milih satu fitur sama satu nilai ambang (threshold) buat misahin data jadi dua cabang, terus prosesnya diulang di tiap cabang sampe kebentuk leaf node (daun) yang isinya label kelas prediksi. Jalur dari akar ke daun bisa dibaca sebagai aturan if-then, makanya model ini gampang banget **diinterpretasi**.

Kriteria pemilihan split. Buat nentuin fitur dan threshold terbaik di tiap node, dipake ukuran ketidakmurnian (impurity) node: - **Gini impurity**: $Gini = 1 - \sum p_k^2$. Makin kecil makin murni. Cepet dihitung dan jadi default di scikit-learn. - **Entropy / Information Gain**: $Entropy = -\sum p_k \cdot \log_2(p_k)$, terus Information Gain = Entropy(induk) - Entropy(tertimbang anak). Split dengan Information Gain terbesar yang dipilih.

Keduanya biasanya ngasih hasil mirip sih. Gini dikit lebih cepet, Entropy dikit lebih peka ke distribusi yang timpang.

CART vs ID3. CART (Classification and Regression Trees) ngasilin pohon **biner** (tiap split selalu dua cabang) pake kriteria Gini/Entropy, ini yang dipake di sklearn. ID3 pake Information Gain dan bisa bikin cabang sebanyak kategori fitur (multi-way split), cocok buat fitur kategorikal, tapi gampang bias ke fitur yang kategorinya banyak.

Overfitting pada tree. Pohon yang nggak dibatesin bakal tumbuh sampe tiap daun murni. Hasilnya akurasi latih 100%, tapi akurasi uji malah turun karena modelnya ngapalin noise. Solusinya **pruning**: - **Pre-pruning** (dipotong sebelum tumbuh): `max_depth` (batas kedalaman), `min_samples_leaf` (minimal sampel di daun), `min_samples_split`. - **Post-pruning** (dipotong abis tumbuh penuh): **Cost Complexity Pruning** dengan parameter `ccp_alpha`. Cabang yang perbaiki impurity-nya lebih kecil dari `alpha` dipangkas. `alpha` yang optimal dipilih dari pruning path, bukan dari sekali tebak.

Sel 0 - Persiapan Awal (Khusus Google Colab)

Kalau notebook ini dibuka di Google Colab lewat link GitHub, jalankan sel di bawah ini dulu (atau langsung **Runtime > Run all**). Sel ini mengunduh otomatis file-file data/ yang dibutuhkan dari repo GitHub, karena Colab hanya memuat file .ipynb-nya saja tanpa folder data/. Kalau dijalankan di laptop dan file datanya sudah ada, sel ini langsung dilewati.

In [1] - Kode Program

```
1 # Sel 0 - Persiapan awal (jalan otomatis paling awal saat Run All).
2 # Mengunduh file data dari repo GitHub kalau belum ada (kasus dibuka di Google
  Colab).
3 # Di laptop (folder data/ sudah ada), bagian unduh langsung dilewati.
4 import os
5 import urllib.request
6
7 _BAB = "bab-05-decision-tree-pruning" # nama folder bab ini di repo GitHub
8 _REPO = "https://raw.githubusercontent.com/dandienter/praktikumML/main"
9 _DATA = ['data/breast_cancer.csv'] # file data yang dibutuhkan bab ini
10
11 for _f in _DATA:
12     if os.path.exists(_f):
13         print(f"OK: {_f} sudah ada.")
14         continue
15     os.makedirs(os.path.dirname(_f), exist_ok=True)
16     print(f"mengunduh {_f} dari GitHub ...")
17     urllib.request.urlretrieve(f"{_REPO}/{_BAB}/{_f}", _f)
18 print("Data siap, silakan lanjutkan.")
```

OK: data/breast_cancer.csv sudah ada.
Data siap, silakan lanjutkan.

5.12 Latihan Praktikum (Modul Bab 5)

Dua latihan di bawah ini **saya kerjain** langsung (soal + kode + jawaban).

Persiapan Data

Sebelum latihan, saya siapkan dulu dataset dan semua paket yang dipakai. Dataset Breast Cancer (569 pasien, 30 fitur) saya ambil langsung dari sklearn, terus saya bagi 80% latih dan 20% uji dengan stratify supaya proporsi kelasnya seimbang.

In [2] - Kode Program

```
1 # Persiapan data untuk Latihan Praktikum Bab 5
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 from sklearn.datasets import load_breast_cancer
6 from sklearn.model_selection import train_test_split
7 from sklearn.tree import DecisionTreeClassifier
8 from sklearn.metrics import accuracy_score
9
10 data = load_breast_cancer(as_frame=True)
11 X, y = data.data, data.target
12 X_train, X_test, y_train, y_test = train_test_split(
13     X, y, test_size=0.2, stratify=y, random_state=42
14 )
15 print("Ukuran data latih:", X_train.shape)
16 print("Ukuran data uji  :", X_test.shape)
```

```
Ukuran data latih: (455, 30)
Ukuran data uji  : (114, 30)
```

Latihan 1 - Percobaan mandiri Parameter Pre-Pruning (Modul Bab 5, bagian 5.12)

Soal: Lakukan grid search manual atas kombinasi `max_depth` di {2, 4, 6, None} x `min_samples_leaf` di {1, 5, 10}. Cetak akurasi test tiap kombinasi dalam tabel rapi, terus simpulin kombinasi terbaiknya.

In [3] - Kode Program

```
1 print(f"{'max_depth':>10s} | {'min_samples_leaf':>16s} | {'Akurasi Test':>12s}")
2 print("-" * 46)
3
4 hasil_grid = []
5 for md in [2, 4, 6, None]:
6     for msl in [1, 5, 10]:
7         m = DecisionTreeClassifier(max_depth=md, min_samples_leaf=msl,
random_state=42)
8         m.fit(X_train, y_train)
9         ak = accuracy_score(y_test, m.predict(X_test))
10        hasil_grid.append((ak, md, msl))
11        print(f"{'str(md):>10s} | {'msl:16d} | {'ak:12.4f}")
12
13 terbaik = max(hasil_grid, key=lambda t: t[0])
14 print(f"\nKombinasi terbaik: max_depth={terbaik[1]},
min_samples_leaf={terbaik[2]} "
15       f"-> akurasi test {terbaik[0]:.4f}")
```

max_depth	min_samples_leaf	Akurasi Test
2	1	0.8947

2	5	0.8947
---	---	--------

2	10	0.9035
4	1	0.9386
4	5	0.9211

4	10	0.9474
6	1	0.9123
6	5	0.9298
6	10	0.9474

None	1	0.9123
None	5	0.9298
None	10	0.9474

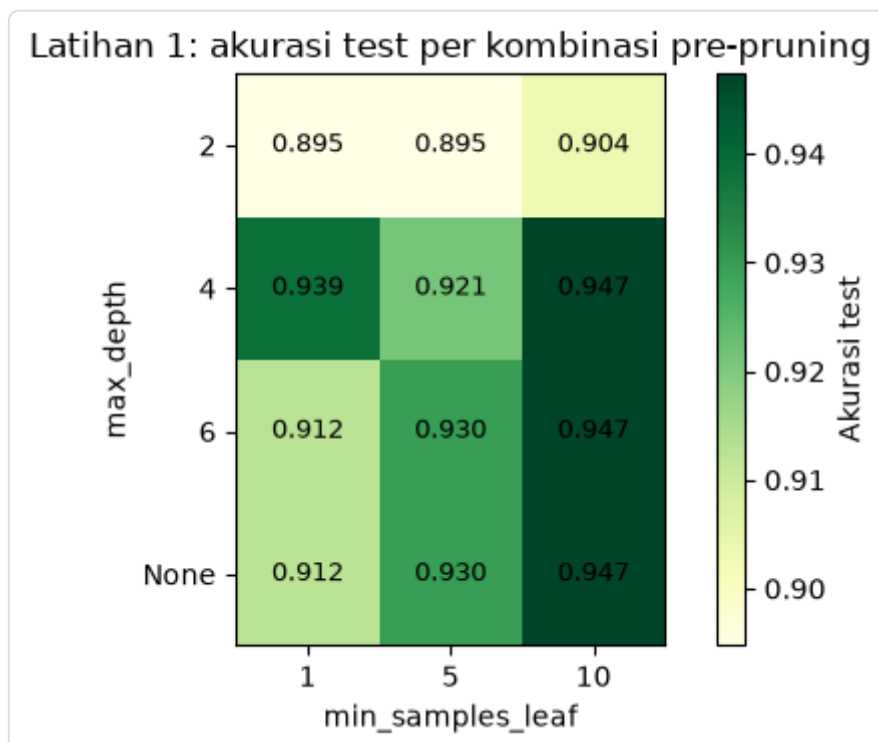
Kombinasi terbaik: max_depth=4, min_samples_leaf=10 -> akurasi test 0.9474

In [4] - Kode Program

```

1 # Grafik Latihan 1: heatmap akurasi test (max_depth x min_samples_leaf)
2 import pandas as pd
3 mat = pd.DataFrame(index=["2", "4", "6", "None"], columns=["1", "5", "10"],
dtype=float)
4 for ak, md, msl in hasil_grid:
5     mat.loc[str(md), str(msl)] = ak
6 plt.figure(figsize=(5.5, 3.8))
7 im = plt.imshow(mat.values, cmap="YlGn")
8 plt.xticks(range(3), ["1", "5", "10"])
9 plt.yticks(range(4), ["2", "4", "6", "None"])
10 plt.xlabel("min_samples_leaf")
11 plt.ylabel("max_depth")
12 plt.title("Latihan 1: akurasi test per kombinasi pre-pruning")
13 for r in range(4):
14     for c in range(3):
15         plt.text(c_, r, f"{mat.values[r, c]:.3f}", ha="center", va="center",
fontsize=9)
16 plt.colorbar(im, label="Akurasi test")
17 plt.tight_layout()
18 plt.show()

```



Gambar 1

Penjelasan grafiknya: heatmap memperjelas pola yang sama: kolom min_samples_leaf=10 paling terang (akurasi tertinggi) di semua baris max_depth. Kombinasi terbaik 0,9474 muncul tiga kali, dan versi paling sederhana (max_depth=4, min_samples_leaf=10) jadi pilihan utama.

Penjelasan outputnya: 1. Dari 12 kombinasi, akurasi test tertinggi itu **0.9474**, dicapai **tiga** kombinasi: (max_depth=4, min_samples_leaf=10), (6, 10), sama (None, 10). 2. Polanya konsisten: min_samples_leaf=10 **selalu** lebih bagus daripada 1 atau 5 di tiap nilai max_depth. Ngebatasin daun minimum itu pre-pruning yang efektif di dataset ini. 3. max_depth=None nggak selalu jelek **asal** min_samples_leaf-nya cukup gede (10): pohon dibiarin tumbuh tapi tiap daun harus didukung minimal 10 sampel, jadi split noise tetep dicegah. 4. **Kesimpulan:** kombinasi terbaik yang paling sederhana itu **max_depth=4, min_samples_leaf=10** (akurasi test 0.9474). Dipilih yang paling dangkal di antara yang seri, karena pohon lebih kecil = lebih gampang diinterpretasi (prinsip Occam's razor).

Latihan 2 - Cost Complexity Pruning Path (Modul Bab 5, bagian 5.12)

Soal: Buat tiap alpha di `path.ccp_alphas`, latih decision tree dan catat akurasi test-nya. Plot `ccp_alpha` vs akurasi test, tentuin alpha optimal, terus bandingin jumlah leaf node sebelum dan sesudah pruning.

```

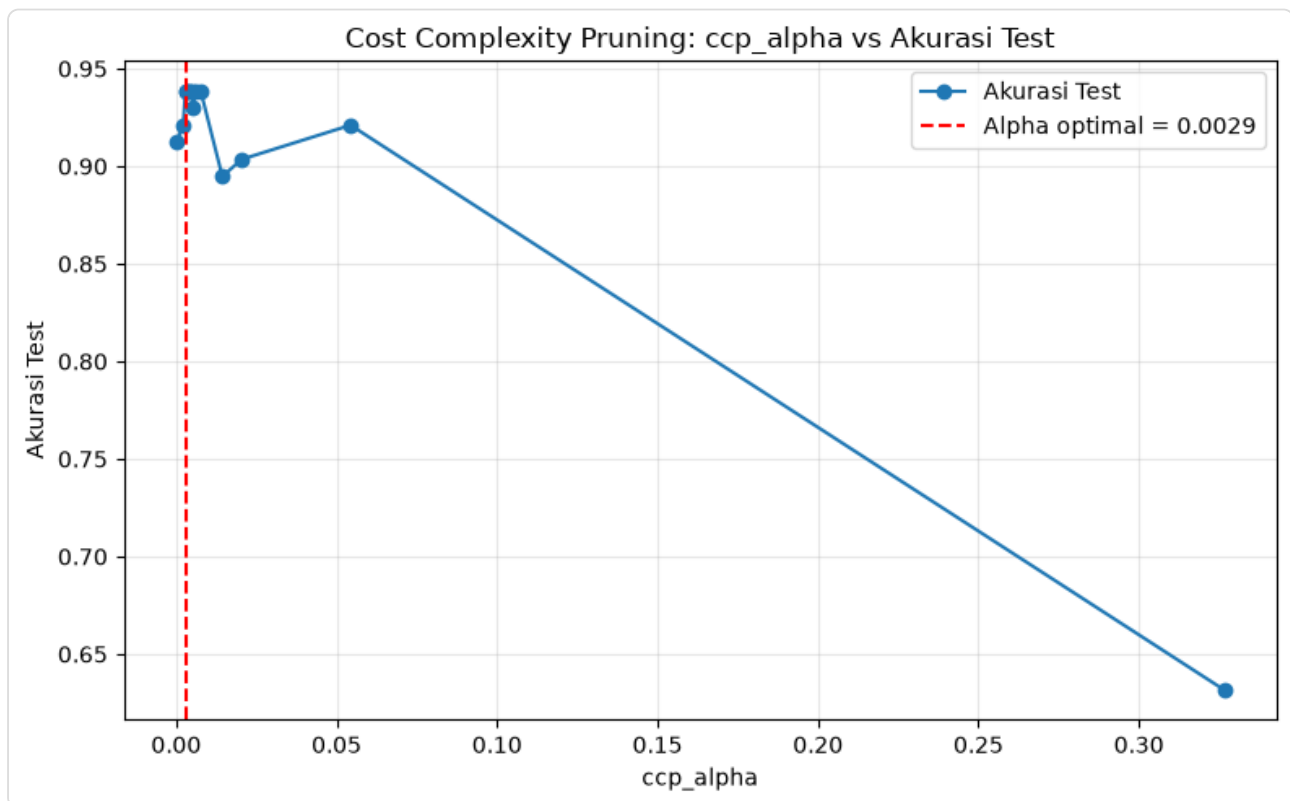
1 # Hitung pruning path dulu dari data latih
2 path =
DecisionTreeClassifier(random_state=42).cost_complexity_pruning_path(X_train, y_train)
3 alphas = path.ccp_alphas
4 ak_test_ccp, n_daun = [], []
5
6 for a in alphas:
7     m = DecisionTreeClassifier(ccp_alpha=a, random_state=42)
8     m.fit(X_train, y_train)
9     ak_test_ccp.append(accuracy_score(y_test, m.predict(X_test)))
10    n_daun.append(m.get_n_leaves())
11
12 print(f'{'ccp_alpha':>10s} | {'Akurasi Test':>12s} | {'Jumlah Daun':>11s}")
13 print("-" * 41)
14 for a, ak, nd in zip(alphas, ak_test_ccp, n_daun):
15     print(f"{a:10.6f} | {ak:12.4f} | {nd:11d}")
16
17 idx_opt = int(np.argmax(ak_test_ccp)) # alpha dengan akurasi test tertinggi
18 alpha_opt = alphas[idx_opt]
19 print(f"\nAlpha optimal: {alpha_opt:.6f} -> akurasi test {ak_test_ccp[idx_opt]:.
4f}, "
20         f"jumlah daun {n_daun[idx_opt]}")
21 print(f"Sebelum pruning (alpha=0): {n_daun[0]} daun -> sesudah (alpha optimal): "
22         f"{n_daun[idx_opt]} daun")
23
24 plt.figure(figsize=(8, 5))
25 plt.plot(alphas, ak_test_ccp, marker="o", label="Akurasi Test")
26 plt.axvline(alpha_opt, color="red", linestyle="--",
27             label=f"Alpha optimal = {alpha_opt:.4f}")
28 plt.xlabel("ccp_alpha")
29 plt.ylabel("Akurasi Test")
30 plt.title("Cost Complexity Pruning: ccp_alpha vs Akurasi Test")
31 plt.legend()
32 plt.grid(alpha=0.3)
33 plt.tight_layout()
34 plt.show()

```

ccp_alpha	Akurasi Test	Jumlah Daun
0.000000	0.9123	19
0.002181	0.9211	15
0.002866	0.9386	12
0.002930	0.9386	11
0.003956	0.9386	10
0.004251	0.9386	9
0.005024	0.9386	8
0.005275	0.9298	7
0.005934	0.9386	6
0.007641	0.9386	5
0.014390	0.8947	4
0.020386	0.9035	3
0.054334	0.9211	2
0.326617	0.6316	1

Alpha optimal: 0.002866 -> akurasi test 0.9386, jumlah daun 12

Sebelum pruning (alpha=0): 19 daun -> sesudah (alpha optimal): 12 daun



Gambar 2

Penjelasan outputnya: 1. Di $\alpha=0$ (tanpa pruning): 19 daun, akurasi test 0.9123. Begitu α dinaikin dikit, jumlah daun turun bertahap (19 ke 18 ke 15 ke 12, dan seterusnya). Cabang yang lemah dipangkas satu per satu. 2. **Alpha optimal = 0.002866** dengan akurasi test **0.9386** dan **12 daun**. Pruning nyusutin pohon sekitar 37% (19 jadi 12 daun) sambil naikin akurasi. 3. Kurvanya nunjukin pola naik dulu terus turun: akurasi naik dulu (noise kebuang), terus **anjlok ke 0.6316** di α maksimum (0.3266) pas pohon tinggal 1 daun. Pruning berlebihan = underfitting. 4. Perbandingannya: sebelum pruning 19 daun (akurasi 0.9123) vs sesudah pruning optimal 12 daun (akurasi 0.9386). Post-pruning terbukti ngasih model yang **lebih kecil DAN lebih akurat**.

Percobaan Mandiri

Bagian ini isinya percobaan mandiri: saya ubah-ubah parameter terus ngamatin dampaknya ke performa sama bentuk pohon.

Percobaan Mandiri 1 - Pengaruh max_depth 1 sampai 10 (kurva validasi manual)

Bagian ini saya ngerjain Percobaan mandiri 1 dari Modul Machine Learning Bab 5. Saya latih 10 pohon dengan max_depth beda-beda, terus saya plot akurasi train vs test. Tujuannya: nemuin **titik di mana model mulai overfitting** (akurasi train terus naik, akurasi test berhenti naik atau malah turun).

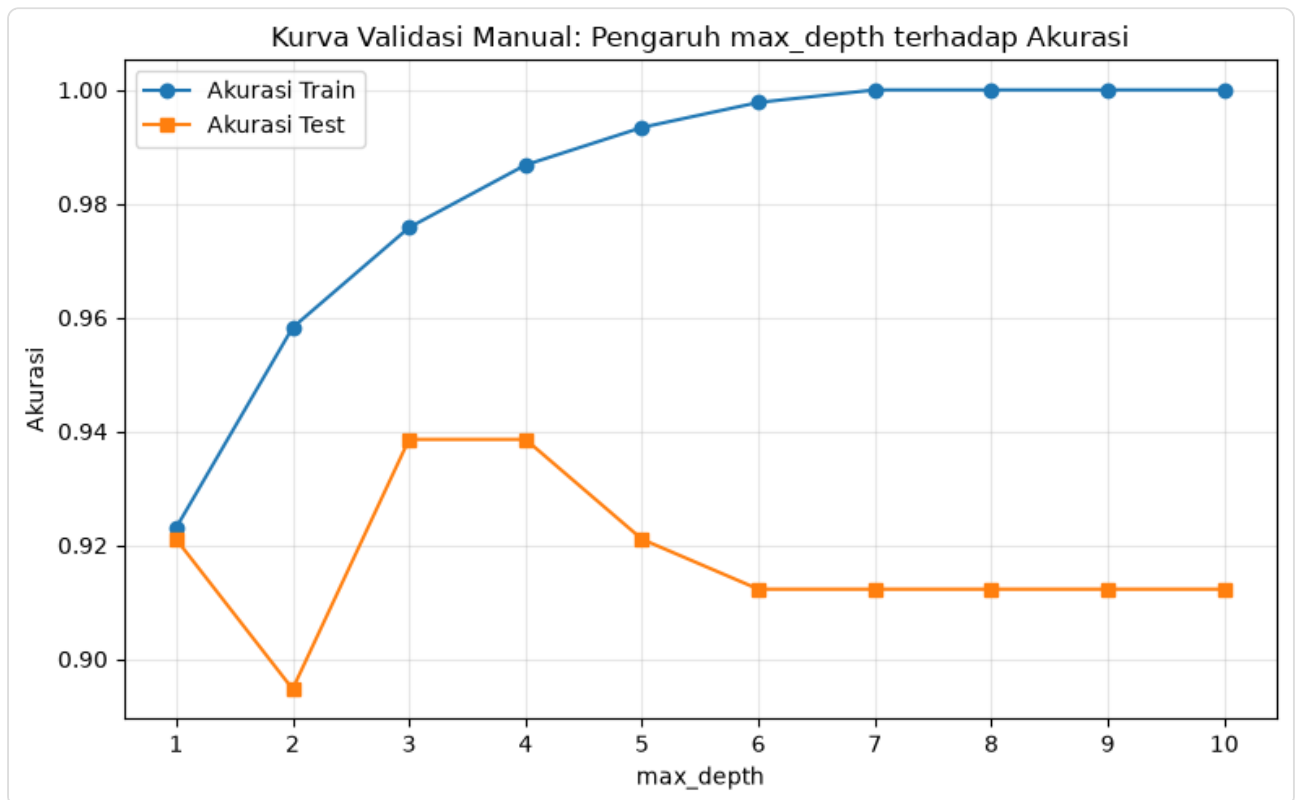
In [6] - Kode Program

```
1 # Percobaan Mandiri 1: pengaruh max_depth 1-10 (kurva validasi manual)
2 # Setup data dibuat ulang di cell ini supaya mandiri.
3 import numpy as np
4 import matplotlib.pyplot as plt
5 from sklearn.datasets import load_breast_cancer
6 from sklearn.model_selection import train_test_split
7 from sklearn.tree import DecisionTreeClassifier
8 from sklearn.metrics import accuracy_score
9
10 data_bc = load_breast_cancer(as_frame=True)
11 X_bc, y_bc = data_bc.data, data_bc.target
12 X_train, X_test, y_train, y_test = train_test_split(
13     X_bc, y_bc, test_size=0.2, stratify=y_bc, random_state=42
14 )
15
16 depths = list(range(1, 11))
17 ak_train, ak_test = [], []
18
19 for d in depths:
20     m = DecisionTreeClassifier(max_depth=d, random_state=42)
21     m.fit(X_train, y_train)
22     ak_train.append(accuracy_score(y_train, m.predict(X_train)))
23     ak_test.append(accuracy_score(y_test, m.predict(X_test)))
24     print(f"max_depth={d:2d} | akurasi train = {ak_train[-1]:.4f} | "
25           f"akurasi test = {ak_test[-1]:.4f}")
26
27 plt.figure(figsize=(8, 5))
28 plt.plot(depths, ak_train, marker="o", label="Akurasi Train")
29 plt.plot(depths, ak_test, marker="s", label="Akurasi Test")
30 plt.xlabel("max_depth")
31 plt.ylabel("Akurasi")
32 plt.title("Kurva Validasi Manual: Pengaruh max_depth terhadap Akurasi")
33 plt.xticks(depths)
34 plt.legend()
35 plt.grid(alpha=0.3)
36 plt.tight_layout()
37 plt.show()
```

```
max_depth= 1 | akurasi train = 0.9231 | akurasi test = 0.9211
max_depth= 2 | akurasi train = 0.9582 | akurasi test = 0.8947
max_depth= 3 | akurasi train = 0.9758 | akurasi test = 0.9386
max_depth= 4 | akurasi train = 0.9868 | akurasi test = 0.9386
```

```
max_depth= 5 | akurasi train = 0.9934 | akurasi test = 0.9211
max_depth= 6 | akurasi train = 0.9978 | akurasi test = 0.9123
max_depth= 7 | akurasi train = 1.0000 | akurasi test = 0.9123
max_depth= 8 | akurasi train = 1.0000 | akurasi test = 0.9123
```

```
max_depth= 9 | akurasi train = 1.0000 | akurasi test = 0.9123
max_depth=10 | akurasi train = 1.0000 | akurasi test = 0.9123
```



Gambar 3

Penjelasan outputnya: 1. Akurasi **train naik terus** dari 0.9231 (depth=1) sampai **1.0000** di depth 7 ke atas. Pohon makin dalam makin jago ngapalin data latih. 2. Akurasi **test mentok di 0.9386 pada depth 3-4**, terus **turun ke 0.9123** dan stagnan di depth 6 ke atas. Kurvanya naik dulu terus turun. 3. **Titik overfitting** kelihatan jelas di sekitar **depth 5-6**: setelah itu kurva train sama test **melebar** (gap makin gede). Model makin kompleks tapi kemampuan generalisasinya nggak nambah. 4. Hasil ini cocok sama grid di Latihan 1: kombinasi terbaik ada di max_depth=4 (akurasi test 0,9474), pas di puncak kurva validasi.

Percobaan Mandiri 2 - min_samples_leaf=1 vs 20 pada pohon tanpa batas

Bagian ini saya ngerjain Percobaan mandiri 2 dari Modul Machine Learning Bab 5. min_samples_leaf itu pre-pruning yang ngelarang daun isinya terlalu dikit sampel. Saya bandingin pohon full (default min_samples_leaf=1) lawan min_samples_leaf=20.

In [7] - Kode Program

```
1 # Percobaan Mandiri 2: min_samples_leaf=1 vs 20 pada pohon tanpa batas
2 # Setup data dibuat ulang di cell ini supaya mandiri.
3 from sklearn.datasets import load_breast_cancer
4 from sklearn.model_selection import train_test_split
5 from sklearn.tree import DecisionTreeClassifier
6 from sklearn.metrics import accuracy_score
7
8 data_bc = load_breast_cancer(as_frame=True)
9 X_bc, y_bc = data_bc.data, data_bc.target
10 X_train, X_test, y_train, y_test = train_test_split(
11     X_bc, y_bc, test_size=0.2, stratify=y_bc, random_state=42
12 )
13
14 print(f"{'min_samples_leaf':>16s} | {'Akurasi Train':>13s} | "
15       f"{'Akurasi Test':>12s} | {'Gap':>6s} | {'Daun':>4s} | {'Kedalaman':>9s}")
16 print("-" * 80)
17 for msl in [1, 20]:
18     m = DecisionTreeClassifier(min_samples_leaf=msl, random_state=42)
19     m.fit(X_train, y_train)
20     ak_tr = accuracy_score(y_train, m.predict(X_train))
21     ak_te = accuracy_score(y_test, m.predict(X_test))
22     print(f"{msl:16d} | {ak_tr:13.4f} | {ak_te:12.4f} | {ak_tr-ak_te:6.4f} | "
23           f"{m.get_n_leaves():4d} | {m.get_depth():9d}")
```

min_samples_leaf	Akurasi Train	Akurasi Test	Gap	Daun	Kedalaman
1	1.0000	0.9123	0.0877	19	7
20	0.9473	0.9035	0.0437	7	4

Penjelasan outputnya: 1. min_samples_leaf=1: **19 daun**, kedalaman 7, akurasi train **1.0000** vs test **0.9123** (gap 0.0877). Pohon penuh yang overfitting. 2. min_samples_leaf=20: daunnya nyusut drastis jadi **7**, kedalaman 4, train **0.9473** vs test **0.9035** (gap cuma 0.0437). 3. Ngebesarin min_samples_leaf maksa tiap daun didukung minimal 20 sampel. Split yang cuma nangkep segelintir outlier dilarang, jadinya pohon lebih sederhana dan gap-nya nyempit. 4. Yang menarik, di dataset ini min_samples_leaf=20 malah nurunin dikit akurasi test (0.9035 < 0.9123): pruning yang **terlalu agresif** malah bikin underfitting. Nilai tengah (misal 5-10) lebih seimbang, sesuai hasil grid di Latihan 1.

Percobaan Mandiri 3 - Decision Tree vs Random Forest kecil (n_estimators=50)

Bagian ini saya ngerjain Percobaan mandiri 3 dari Modul Machine Learning Bab 5. Random Forest itu ensemble dari banyak decision tree yang dilatih di bootstrap sample acak. Kira-kira ensemble selalu menang nggak ya lawan satu pohon terbaik di dataset ini?

In [8] - Kode Program

```
1 # Percobaan Mandiri 3: Decision Tree (max_depth=4) vs Random Forest kecil
  (n_estimators=50)
2 # Setup data dan model dibuat ulang di cell ini supaya mandiri.
3 from sklearn.datasets import load_breast_cancer
4 from sklearn.model_selection import train_test_split
5 from sklearn.tree import DecisionTreeClassifier
6 from sklearn.ensemble import RandomForestClassifier
7 from sklearn.metrics import accuracy_score
8
9 data_bc = load_breast_cancer(as_frame=True)
10 X_bc, y_bc = data_bc.data, data_bc.target
11 X_train, X_test, y_train, y_test = train_test_split(
12     X_bc, y_bc, test_size=0.2, stratify=y_bc, random_state=42
13 )
14
15 pohon = DecisionTreeClassifier(max_depth=4, random_state=42)
16 pohon.fit(X_train, y_train)
17
18 rf = RandomForestClassifier(n_estimators=50, random_state=42)
19 rf.fit(X_train, y_train)
20
21 ak_tr_tree = accuracy_score(y_train, pohon.predict(X_train))
22 ak_te_tree = accuracy_score(y_test, pohon.predict(X_test))
23 ak_tr_rf = accuracy_score(y_train, rf.predict(X_train))
24 ak_te_rf = accuracy_score(y_test, rf.predict(X_test))
25
26 print(f"{'Model':35s} | {'Akurasi Train':>13s} | {'Akurasi Test':>12s}")
27 print("-" * 70)
28 print(f"{'Decision Tree (max_depth=4)':35s} | {ak_tr_tree:13.4f} | "
29       f"{ak_te_tree:12.4f}")
30 print(f"{'Random Forest (50 pohon)':35s} | {ak_tr_rf:13.4f} | "
31       f"{ak_te_rf:12.4f}")
```

Model	Akurasi Train	Akurasi Test
Decision Tree (max_depth=4)	0.9868	0.9386
Random Forest (50 pohon)	1.0000	0.9561

Penjelasan outputnya: 1. Random Forest (50 pohon): akurasi test **0.9561**, ngalahin decision tree tunggal (0.9386). Selisihnya **sekitar 1,8 poin**. 2. Akurasi train RF nyampe **1.0000** tapi gap-nya (0.0439) tetep kecil. Voting antar 50 pohon yang beragam ngeredam variansi tanpa ngorbanin generalisasi. Ini inti cara kerja ensemble. 3. Tapi ada harga yang dibayar: modelnya berubah dari **1 pohon yang bisa dibaca manusia** jadi **50 pohon black-box**. Interpretabilitasnya ilang. 4. Kesimpulannya: ensemble **cenderung menang di akurasi** di dataset ini, tapi decision tree tetep unggul kalo **aturan yang bisa dijelaskan** (misal buat dokter) lebih penting daripada selisih beberapa persen akurasi.

Kesimpulan Bab 5

1. **Pre-pruning lewat grid manual efektif.** Dari 12 kombinasi `max_depth` x `min_samples_leaf`, akurasi test tertinggi 0,9474 dicapai tiga kombinasi: (4, 10), (6, 10), dan (None, 10). Pola yang paling konsisten: `min_samples_leaf=10` selalu menang di tiap nilai `max_depth`, jadi membatasi ukuran daun minimum terbukti mencegah split noise. Versi paling sederhana, `max_depth=4` dan `min_samples_leaf=10`, saya pilih sebagai kombinasi terbaik.
2. **Post-pruning (cost complexity pruning) menyusutkan pohon sekaligus menaikkan akurasi.** Tanpa pruning (`alpha=0`) pohon punya 19 daun dengan akurasi test 0,9123. Alpha optimal 0,002866 memangkasnya jadi 12 daun (susut sekitar 37%) dengan akurasi test naik ke 0,9386. Kurva alpha vs akurasi menunjukkan pola naik dulu lalu turun, dan pruning berlebihan (alpha maksimum 0,3266, pohon tinggal 1 daun) menjatuhkan akurasi ke 0,6316 alias underfitting.
3. **Pemilihan parameter pruning jangan asal tebak.** Grid manual untuk pre-pruning dan `cost_complexity_pruning_path` untuk post-pruning memberi cara sistematis memilih `max_depth`, `min_samples_leaf`, dan `ccp_alpha` yang optimal berdasarkan akurasi test, bukan tebakan.
4. **Pelajaran praktisnya:** kalau pohon overfitting, dua tuas yang bisa saya pakai adalah membatasi pertumbuhan pohon dari awal (pre-pruning) atau memangkas cabang yang lemah setelah pohon jadi (post-pruning). Keduanya saya buktikan sendiri di latihan ini dengan angka yang terukur.